

Week 4 — Kafka Producer & Consumer in Java

Apache Kafka with the Java Client API · com.example.kafka

Java

Apache Kafka

Docker Compose

WEEK 4 — TASK

Write a simple Java program to create a Kafka producer and produce messages to a topic.

This project sets up a local single-broker Kafka cluster with Docker, then uses the plain Kafka Java client to create a topic, produce a message to it, and consume that message back — all from the command line.

1. Project Structure

```
KafkaJavaProject/
├── lib/
│   ├── kafka-clients-3.7.0.jar
│   ├── slf4j-api-2.0.7.jar
│   └── slf4j-simple-2.0.7.jar      # optional (removes SLF4J warnings)
├── src/
│   └── main/java/com/example/kafka/
│       ├── KafkaTopicCreator.java
│       ├── KafkaProducerExample.java
│       └── KafkaConsumerExample.java
└── out/                          # compiled .class files (created by javac)
```

2. Start Kafka (Docker Compose)

Spin up a single-node Kafka broker with Zookeeper using the compose file below, then run `docker compose up -d`.

docker-compose.yml

```
version: '3'
services:
  zookeeper:
    image: wurstmeister/zookeeper:latest
    container_name: zookeeper
    ports:
      - "2181:2181"

  kafka:
    image: wurstmeister/kafka:latest
    container_name: kafka
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"
    depends_on:
      - zookeeper
```

3. Java Source Files

Broker is exposed on `localhost:9092`. All three classes read input from the console with `Scanner` and validate the topic name.

3.1 KafkaTopicCreator.java

KafkaTopicCreator.java

```

package com.example.kafka;

import org.apache.kafka.clients.admin.AdminClient;
import org.apache.kafka.clients.admin.AdminClientConfig;
import org.apache.kafka.clients.admin.NewTopic;

import java.util.Collections;
import java.util.Properties;
import java.util.Scanner;

public class KafkaTopicCreator {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter topic name: ");
        String topicName = scanner.nextLine().trim();

        // Validate topic name
        if (topicName.isEmpty() || !topicName.matches("[a-zA-Z0-9._-]+")) {
            System.out.println(
                "Invalid topic name. Use only letters, numbers, dots (.), underscores (_), or hyphens (-).");
            scanner.close();
            return;
        }

        Properties props = new Properties();
        // Kafka is currently exposed on localhost:9092
        props.put(
            AdminClientConfig.BOOTSTRAP_SERVERS_CONFIG,
            "localhost:9092"
        );

        try (AdminClient adminClient = AdminClient.create(props)) {
            NewTopic newTopic = new NewTopic(
                topicName,
                1,
                (short) 1
            );

            adminClient
                .createTopics(Collections.singletonList(newTopic))
                .all()
                .get();

            System.out.println(
                "Topic \"" + topicName + "\" created successfully!"
            );
        } catch (Exception e) {
            if (e.getCause() != null) {
                System.out.println(
                    "Topic creation failed: " + e.getCause().getMessage()
                );
            } else {
                System.out.println(
                    "Topic creation failed: " + e.getMessage()
                );
            }
        } finally {
            scanner.close();
        }
    }
}

```

3.2 KafkaProducerExample.java — the Week 4 program

This is the core Week 4 deliverable: it builds a `KafkaProducer<String, String>`, reads a topic name and a message from the console, wraps them in a `ProducerRecord`, and sends the message synchronously (blocking on `.get()` to confirm delivery).

KafkaProducerExample.java

```

package com.example.kafka;

import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerConfig;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.apache.kafka.common.serialization.StringSerializer;

import java.util.Properties;
import java.util.Scanner;
import java.util.concurrent.ExecutionException;

public class KafkaProducerExample {

    public static void main(String[] args) {
        Properties props = new Properties();
        // Kafka is currently running on localhost:9092
        props.put(
            ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
            "localhost:9092"
        );
    }
}

```

```

    );
    props.put(
        ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
        StringSerializer.class.getName()
    );
    props.put(
        ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
        StringSerializer.class.getName()
    );

    try {
        KafkaProducer<String, String> producer =
            new KafkaProducer<>(props);
        Scanner scanner = new Scanner(System.in)
    } {
        System.out.print("Enter topic name: ");
        String topic = scanner.nextLine().trim();

        // Validate topic name
        if (topic.isEmpty() ||
            !topic.matches("[a-zA-Z0-9._-]+")) {
            System.out.println(
                "Invalid topic name. Use only letters, numbers, " +
                "dots (.), underscores (_), or hyphens (-)."
            );
            return;
        }

        System.out.print("Enter message: ");
        String message = scanner.nextLine();

        ProducerRecord<String, String> record =
            new ProducerRecord<>(topic, message);

        try {
            // Wait for Kafka to confirm delivery
            producer.send(record).get();
            System.out.println(
                "Message sent successfully to topic \"" +
                topic + "\""
            );
        } catch (InterruptedException e) {
            System.out.println(
                "Failed to send message: " +
                e.getCause().getMessage()
            );
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            System.out.println(
                "Message sending was interrupted."
            );
        }
    } catch (Exception e) {
        System.out.println(
            "Unexpected error: " + e.getMessage()
        );
    }
}
}

```

3.3 KafkaConsumerExample.java

KafkaConsumerExample.java

```

package com.example.kafka;

import org.apache.kafka.clients.consumer.ConsumerConfig;
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.clients.consumer.ConsumerRecords;
import org.apache.kafka.clients.consumer.KafkaConsumer;
import org.apache.kafka.common.serialization.StringDeserializer;

import java.time.Duration;
import java.util.Collections;
import java.util.Properties;
import java.util.Scanner;

public class KafkaConsumerExample {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter topic name: ");
        String topic = scanner.nextLine().trim();

        // Validate topic name
        if (topic.isEmpty() ||
            !topic.matches("[a-zA-Z0-9._-]+")) {
            System.out.println(
                "Invalid topic name. Use only letters, numbers, " +
                "dots (.), underscores (_), or hyphens (-)."
            );
        }
    }
}

```

```

        scanner.close();
        return;
    }

    Properties props = new Properties();
    // Kafka is currently running on localhost:9092
    props.put(
        ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG,
        "localhost:9092"
    );
    // Consumer group ID
    props.put(
        ConsumerConfig.GROUP_ID_CONFIG,
        "test-group"
    );
    // Key deserializer
    props.put(
        ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
        StringDeserializer.class.getName()
    );
    // Value deserializer
    props.put(
        ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
        StringDeserializer.class.getName()
    );
    // Read from beginning if this consumer group has no offset
    props.put(
        ConsumerConfig.AUTO_OFFSET_RESET_CONFIG,
        "earliest"
    );

    try (KafkaConsumer<String, String> consumer =
        new KafkaConsumer<>(props)) {
        consumer.subscribe(Collections.singletonList(topic));

        System.out.println("Kafka Consumer started.");
        System.out.println(
            "Waiting for messages on topic \"" +
            topic + "\"..."
        );

        // Continuously poll for messages
        while (true) {
            ConsumerRecords<String, String> records =
                consumer.poll(Duration.ofMillis(1000));

            for (ConsumerRecord<String, String> record : records) {
                System.out.printf(
                    "Received message: key=%s, value=%s, " +
                    "partition=%d, offset=%d%n",
                    record.key(),
                    record.value(),
                    record.partition(),
                    record.offset()
                );
            }
        }
    } catch (Exception e) {
        System.err.println("Error in Kafka Consumer:");
        e.printStackTrace();
    } finally {
        scanner.close();
    }
}

```

4. Compile

```

javac -cp "lib/*" -d out src/main/java/com/example/kafka/*.java
java -cp "out:lib/*" com.example.kafka.KafkaTopicCreator
java -cp "out:lib/*" com.example.kafka.KafkaConsumerExample
java -cp "out:lib/*" com.example.kafka.KafkaProducerExample

```

5. Run — Three Terminals

TERMINAL 1 — CREATE THE TOPIC

```

$ java -cp "out:lib/*" com.example.kafka.KafkaTopicCreator
Enter topic name: test-topic

```

```
Topic "test-topic" created successfully!
```

TERMINAL 2 — START THE CONSUMER (LEAVE RUNNING)

```
$ java -cp "out:lib/*" com.example.kafka.KafkaConsumerExample  
Enter topic name: test-topic
```

TERMINAL 3 — RUN THE PRODUCER

```
$ java -cp "out:lib/*" com.example.kafka.KafkaProducerExample  
Enter topic name: test-topic  
Enter message: Hello Kafka
```

```
Message sent successfully to topic "test-topic"
```

6. Expected Consumer Output

Back in Terminal 2, the running consumer should print:

```
Received message: key=null, value=Hello Kafka, partition=0, offset=0
```